POS approval and evidence verifier contract

This is a supplementary, reviewable machine-verification format. It does not change the frozen business contract, approve a decision, or establish a production trust authority. No trust key, signature, actual approval, provider result or physical test result was created for this review.

Why a supplement is necessary

The original gate accepted invented identity/signature references and any 64-character digest without opening or verifying signature evidence. A hash only identifies bytes; it does not authenticate an approver. The new verifier checks an **issuer attestation** using Ed25519. The independently trusted issuer remains responsible for checking the real approver's identity, role, authority, fresh authentication, decision and supporting evidence. This local script does not directly prove a human's identity, provider behavior, expert qualification or hardware performance.

Security/IAM and CI operators must select and approve the authentic approval/evidence issuer and its adapter to this envelope before use. Private signing keys remain in that issuer's managed custody. This review supplies verification code and negative/positive synthetic tests, not that organizational trust decision.

Independent trust configuration

Set `POS_GATE_TRUSTED_PUBLIC_KEYS_PATH` to an absolute path **outside the candidate workspace** containing a JSON object keyed by issuer key ID. A key entry has:

```
{
  "approved-issuer-key-id": {
    "publicKeyPem": "<operator-provisioned Ed25519 SPKI public key>",
    "evidenceClasses": ["ARTIFACT_BOUND_APPROVAL"],
    "validFrom": "<UTC ISO timestamp>",
    "validUntil": "<UTC ISO timestamp>"
  }
}
```

The file is managed by the operator/CI trust boundary, never accepted from an evidence envelope. Missing or invalid configuration yields a blocked diagnostic. Do not trust keys generated by unit tests. Public keys are not secrets, but their authenticity, class permissions, validity and removal/revocation must be controlled. Remove a key to revoke its acceptance. Environment/configuration authority must be restricted outside untrusted candidate execution.

Envelope and canonical signature

The existing `signatureReference` or program `verificationReference` points to a repository-relative JSON evidence file. Absolute paths, external URLs, traversal outside the workspace, and symlink escapes are rejected; the gate performs no network fetch. Export externally verified evidence into a redacted local bundle when its authoritative origin is remote.

```
{
  "schemaVersion": "1.0.0",
  "keyId": "<approved issuer key ID>",
  "payload": {
    "result": "VERIFIED",
    "binding": "<object generated as described below>",
    "evidenceClass": "ARTIFACT_BOUND_APPROVAL",
    "verifiedAt": "<UTC ISO timestamp>",
    "reviewOrExpiryAt": "<UTC ISO timestamp>",
    "artifacts": [{"path": "<redacted supporting artifact>", "sha256": "<actual lowercase SHA-
256>"}]
  },
  "signature": "<base64 Ed25519 signature over canonical payload UTF-8>"
}
```

Canonicalization is `canonicalJson()` exported from `scripts/pos-governance-evidence.js`: recursively sort object keys lexicographically, preserve array order, serialize primitives with `JSON.stringify`, and use no extra whitespace. Sign UTF-8 bytes of the **payload only**. The register's digest hashes the actual complete envelope file bytes, independently of the signature. Every supporting artifact must exist inside the workspace and match its hash. Times must be valid UTC ISO timestamps, not future dated or expired, and within the issuer key's validity interval. A signed envelope's expiry cannot exceed the decision expiry.

G1 decision approvals

Preserve the current approval register schema. Populate all its existing decision fields from the accountable review; `effectiveVersion` must equal the frozen contract version. One authenticated approval is needed for each required role. Fresh authentication must precede approval by no more than ten minutes; the issuer's verification time must be on or after approval.

For each approval, obtain its exact `payload.binding` from exported `approvalBinding(contractBinding, decisionRecord, approval)` in `scripts/pos-g1-contract-gate.js`. `contractBinding` is the exact `evidenceVerification.binding` in the fresh G1 report and contains artifact ID, version, path, full file hash and current authorized release scope. The decision portion binds ID, selected option, rationale, effective version, review/expiry, evidence links, affected capabilities and rollback policy. The approver portion binds identity, role, authority reference and both authentication/approval times. Signature reference and digest are omitted from the signed binding to avoid a cycle; place them in the existing approval fields after signing.

All record fields, selected option, contract bytes and release scope must still match at verification. Reusing an old signature after changing any bound field fails. Material business changes require a new reviewed contract version and new applicable approvals; never silently edit the frozen artifact.

Program external evidence and gate closure

Program records are unchanged in this review. When authentic evidence is available, an external record additionally supplies `verificationReference` and `verificationEvidenceSha256`; marking `status: VERIFIED` alone has no effect. Build its signed binding using `programEvidenceBinding(program, gate, contractSha256, externalItem)` exported from `scripts/pos-enterprise-program-gate.js`. It binds program ID, gate ID, contract hash, exact candidate object, the hash of dependencies/required evidence/stop conditions/external

requirements/work-package IDs, external evidence ID, and its required scope. The candidate HEAD must equal the current checkout HEAD. The old recorded candidate currently does not match.

For a gate declared `PASSED`, supply `closureEvidence: {reference, sha256}`. Build the binding with `programEvidenceBinding(program, gate, contractSha256)` (omit the external item). Closure attestations use `ARTIFACT_BOUND_APPROVAL` and additionally include a nonempty signed `payload.sourceManifest` array of `{path, sha256}` for the source/config/test files reviewed. Each file must still match. The issuer must establish manifest completeness for the gate and attest every required scenario and stop condition; a matching partial manifest alone is not release proof. Referenced artifacts contain the exact test runs, invariant results, reviewer decisions and unresolved finding disposition. Work-package status strings and source-text markers alone do not establish closure.

External evidence classes are constrained by ID:

Evidence item	Allowed signed class
EXT-G1-APPROVALS	Derived from actual G1 verification; no duplicate approval envelope needed
EXT-PROVIDER-SANDBOX	PROVIDER_SANDBOX
EXT-CM-QUALIFIED-REVIEW	QUALIFIED_HUMAN_REVIEW
EXT-HARDWARE-MATRIX	PHYSICAL_HARDWARE, or ARTIFACT_BOUND_APPROVAL for an explicit signed exclusion
EXT-CLOSE-CYCLE	PRODUCTION_LIKE_OPERATIONS
EXT-INVENTORY-POLICY	ARTIFACT_BOUND_APPROVAL
EXT-AR-RETURNS-POLICY	ARTIFACT_BOUND_APPROVAL
EXT-OPS-DRILLS	PRODUCTION_LIKE_OPERATIONS
EXT-PILOT-COHORTS	CONTROLLED_PILOT

Local unit/static/database tests cannot substitute for these classes. The issuer must bind provider/merchant/environment/scenario scope, qualified reviewer/checker scope, device/firmware/OS/browser matrix or exclusion, close-cycle identifiers, operating-environment/drill scope, or pilot/cohort/expansion boundaries in the supporting evidence as appropriate. The verifier establishes signature and scope binding; those real-world facts still require authentic source evidence and accountable review.

Each external payload also requires a signed `verificationScope` object:

Evidence item	Required scope fields
Provider sandbox	`providerId`, redacted `merchantAccountId`, `environment: SANDBOX`
Cameroon review	`country: Cameroon`, distinct nonempty `reviewerId` and `checkerId`
Physical hardware	`disposition: TESTED`, `matrixReference`; supporting artifact contains the exact tested matrix
Signed hardware exclusion	`disposition: EXCLUDED`, nonempty `excludedCapabilities`, `rationale`; cannot be represented as physical certification
Close cycle	`environment: PRODUCTION_LIKE`, `cycleId`
Inventory / AR policies	`policyVersion`, `ownerId`
Operational drills	`environment: PRODUCTION_LIKE`, nonempty `drillIds`

Evidence item	Required scope fields
Pilot cohorts	Nonempty `cohortIds`, nonempty `stages`, `decisionReference`

These identifiers must be redacted stable references, not secrets. Structural presence alone does not establish their authenticity; the trusted issuer verifies their authoritative source and the full scenarios in the attached artifacts.

Canonical gate dependencies and the complete nine-item external register cannot be removed or duplicated to obtain a pass. Independent evidence blockers remain visible even where upstream dependencies block progression. Passing this tooling never authorizes production activation; the separate controlled release decision remains required.